

Universidad Tecnológica de Bolívar

Parcial ingeniería de software

Profesor: Aisner Jose Marrugo Juliao

Integrantes:

Josh Robinson Ortega Castellón

Luis Eduardo Mendoza Angulo

Fundamentos

Problemas, actores y alcance

Un colectivo de escritores prepara una ficción interactiva para un festival universitario y no sabe programar. Hoy, cambiar una escena o una decisión exigiría modificar el programa. Además, un error en el contenido podría cerrar la aplicación frente al lector. Se necesita un motor que lea narraciones desde archivos, muestre solo las opciones compatibles con lo ocurrido, registre las consecuencias de cada elección y avise al escritor qué corregir cuando un archivo es inválido.

Actores

Actor	**Rol**	**Interés principal**
Escritora principal	Edita los archivos de historia	Cambiar escenas y decisiones sin tocar código
Diseñador de la experiencia	Define condiciones y consecuencias	Opciones que aparecen según lo descubierto, o sea decisiones con peso
Coordinadora del festival	Responsable de la puesta en marcha	Errores diagnosticables, por ejemplo, que el programa no cierre
Lector	Juega la historia	Leer, elegir, llegar a un final y reiniciar

Alcance

Dentro (v1)	Fuera (declarado por el cliente)
Carga de historias desde archivo	Cuentas, guardado entre ejecuciones, base de datos
Condiciones de igualdad y efectos sobre variables booleanas	Lenguaje de expresiones, editor visual, audio, imagenes
Opciones filtradas por estado, finales y reinicio	Conexión a internet
Validación de contenido con mensajes de causo y campo	Análisis automatico de todos los caminos posibles
Historia de referencia (escenas) y segunda historia (≥ 3 escenas)	Retroceso de decisiones (conflicto abierto, sin implementar)

Dos riesgos con mitigación

ID	Riesgo	Prob.	Impacto	Mitigación	Como sabremos que ocurre
R1	**Acoplamiento del motor a la historia de referencia.** Si se programan reglas específicas para E1, E3, etc., la segunda historia y el cambio de la hora 6 rompen el motor.	Media	Alto	Contenido, reglas e interfaz en módulos separados. Condiciones y efectos evaluados solo desde los datos. Probar desde temprano con la segunda historia.	Un identificador de escena aparece en el código fuente del motor.
R2	**Cambio tardío con tiempo limitado.** La solicitud llega en la hora 6, con la v1 posiblemente incompleta, y puede afectar requisitos, modelos y pruebas a la vez.	Alta	Alto	Guardar una versión inicial en la hora 6 con etiqueta en Git. Mantener trazabilidad desde el comienzo. Reservar 1 h 30 min para cambio y pruebas. Dejar IDs de RF libres (RF20+) para el cambio.	Backlog sin tarea de cambio al llegar a la hora 7.

Priorización de atributos de calidad

Prioridad	Atributo	Justificación (voz del cliente)	Se medirá con
1	Modificabilidad del contenido	La escritora quiere agregar escenas sin pedir cambios al programa	Una historia nueva funciona sin edtiar el motor
2	Confiabilidad/robustez ante errores	La coordinadora no acepta cierres inesperados	Entradas inválidas o archivos defectuosos no terminan el programa
3	Claridad de los errores (usabilidad para el escritor)	Debe saber qué corregir	El mensaje indica causa y campo o ID
4	Testeabilidad	Es exigida por la entrega e indispensable para el cambio	Cobertura de los escenarios U1-U5, I1 y el nuevo

Baja	Rendimiento, seguridad portabilidad	Historias pequeñas, uso local, sin red ni cuentas	No se mide en esta versión

Requerimientos

ID	Necesidad	Requisito	Criterio de aceptación
RF01	N01	Cargar una historia desde un archivo indicado, sin modificar código para cambiar de historia.	Dados la historia de referencia y la segunda historia, ambas se cargan y muestran su título.
RF02	N01	Iniciar la partida en inicio con las variables iniciales.	La partida comienza en E1 con ambas variables en falso.
RF03	N02	Mostrar el texto de la escena actual.	En E1 se muestra el texto de “La señal”.
RF04	N02	Mostrar solo las opciones cuyas condiciones se cumplen todas; sin condiciones, siempre se muestra.	En E3 con conoce_origen = verdadero se ve “Activar con código” y no “Activar sin verificar” (CA01).
RF05	N03	Aplicar los efectos de la opción elegida y llevar a su destino.	En E3 con origen conocido, “Activar con código” deja confia_en_la_voz = verdadero y va a E4.
RF06	N03	Rechazar una opción inexistente o no habilitada, aunque se envíe su ID al motor, sin alterar el estado.	En E3 sin origen, enviar “Activar con código” se rechaza y sigue en E3 (CA02).
RF07	N03	Tratar una entrada de lector no válida como error recuperable, sin alterar la partida.	Ingresar "abc" o "7" muestra un mensaje y vuelve a pedir la elección.
RF08	N04	Conservar las variables entre escenas y evaluar condiciones con el estado acumulado.	E1→E2 (investigar)→E3 muestra “Activar con código”; con “continuar” muestra “Activar sin verificar”.
RF09	N05	Mostrar el texto de una escena final y avisar que la partida terminó, sin opciones.	En E4, E5 o E6 se muestra el texto y el aviso de fin.
RF10	N05	Permitir llegar a los tres finales según las decisiones.	Las tres rutas desde E3 alcanzan E4, E5 y E6.
RF11	N05	Informar el bloqueo cuando una escena no final no tenga opciones habilitadas, sin cerrarse.	Una escena de prueba con opciones imposibles

			muestra el aviso de bloqueo.
RF12	N05	Permitir reiniciar tras un final o bloqueo, restaurando escena y variables iniciales.	Tras terminar en E4, reiniciar deja E1 y variables en falso.
RF13	N06	Rechazar antes de jugar un archivo con alguno de los 9 defectos de la tabla inferior.	Cada archivo defectuoso es rechazado.
RF14	N06	Indicar en cada rechazo la causa y el campo o ID involucrado.	Destino E99 en la opción investigar de E2 aparece en el mensaje.
RF15	N06	No crear partida con un archivo rechazado; terminar la carga de forma controlada.	Con destino E99 no hay partida ni cierre inesperado (CA03).

Requerimientos no funcionales

ID	Enunciado	Métrica y umbral	Procedimiento y evidencia
RNF01 Modificabilidad	Cambiar de historia o editar escenas, opciones, condiciones y efectos modificando solo el archivo de contenido.	0 archivos de código modificados; 0 IDs E1 a E6 en el código del motor.	git diff --stat vacío tras cargar la segunda historia; prueba test_motor_sin_ids_de_historia. Evidencia: evidencias/rnf01.txt.
RNF02 Confiabilidad	Ante archivos defectuosos o entradas inválidas, terminar de forma controlada, sin *traceback* ni cierre inesperado.	18 de 18 casos (9 defectos de archivo + 9 entradas inválidas).	Prueba parametrizada test_rnf02_sin_excepciones y ejecución por subproceso sin la palabra Traceback. Evidencia: evidencias/pytest_ultima_ejecucion.txt.
RNF03 Claridad de errores	Todo mensaje de rechazo incluye causa y campo o ID, en español y sin nombres internos del código.	9 de 9 mensajes con causa e ID; 0 con nombres de clases o excepciones.	Prueba test_rnf03_mensajes_claros y revisión cruzada entre integrantes. Evidencia: evidencias/rnf03_revision.md.

Preguntas y supuestos

#	Tipo	Contenido
1	Supuesto	Se informan todos los errores del archivo, no solo el primero.
2	Pregunta al cliente	¿El retroceso debe estar siempre disponible o solo en ciertas decisiones? Acuerdo provisional: sin retroceso en la v1.
3	Supuesto	Las opciones se muestran en el orden del archivo.
4	Supuesto	El lector elige por número; la interfaz lo traduce al ID de la opción.

Defecto detectado en la revisión cruzada

Defecto: La palabra aparecer es ambigua al describir las opciones disponibles.

Corrección: Se reemplazó por “se muestra”, y RF04 define que solo se muestran las opciones cuyas condiciones se cumplen todas

Procesos

Proceso elegido y alternativa comparada

Proceso elegido: desarrollo incremental con tablero Kanban. Se construye la solución en incrementos pequeños y verificables (carga, motor, interfaz, validación, pruebas). Cada incremento pasa por un tablero con límite de trabajo en curso. El cambio de la hora 6 entra como una tarea más, sin romper el flujo.

Alternativa comparada: Scrum. Se trabaja en sprints de duración fija, con un *sprint backlog* comprometido al inicio, y con eventos como planificación, reunión diaria,

Criterio	Incremental+Kanban(elegido)	Scrum (descartada)
Cambio en la hora 6	Entra como tarjeta nueva en cualquier momento	El sprint en curso se protege de cambios; habría que replanificar o cancelar el sprint
Duración y cadencia	Flujo continuo, sin iteraciones fijas	Requiere sprints cortos (ej.: 3 h), con planificación y revisión en cada uno
Carga de ceremonias	Mínima: tablero y límite de trabajo en curso	Eventos y roles (Scrum Master, Product Owner) que no se justifican en una pareja y 12 h
Control del trabajo	Límite de 2 tareas por persona	Capacidad fijada por sprint

Por qué se elige Kanban y no Scrum: el parcial dura 12 horas, lo realiza una pareja y contiene un cambio programado. Scrum añadiría ceremonias y roles que consumirían tiempo, y su compromiso de sprint choca con una solicitud que llega a mitad del trabajo. Kanban permite absorber el cambio como una tarea más y mantener siempre una versión ejecutable.

Como se realiza cada actividad del proceso

Actividad	Como se hará en esta entrega	Evidencia
Especificación	Se redactan RF y RNF desde las necesidades N01 a N06, con criterios de aceptación. Se actualizan al llegar el cambio (RF20+).	Informe, Unidad 4; matriz de trazabilidad
Desarrollo	Incrementos en orden: modelo y carga, motor de reglas, interfaz de consola, validación. Arquitectura de monolito modular (ver 2.3).	Repositorio Git con commits por incremento
Validación	Pruebas automatizadas (U1 a U5, I1), verificación de los tres RNF y recorrido manual de los tres finales.	`evidencias/pytest_ultima_ejecucion.txt`, video
Evolución	En la hora 6: etiqueta Git de la versión inicial, análisis de impacto y cambio. Se actualizan requisitos, modelos, pruebas y trazabilidad.	Etiqueta `v1-hora6`, sección de gestión del cambio

Ajuste de arquitectura

Pasamos a un **monolito modular** en lugar de capas con despliegue separado. Es un solo programa Python con módulos de responsabilidad clara, sin servicios aparte ni infraestructura. Se justifica porque el proyecto es pequeño y el tiempo es corto. Los módulos siguen aislando contenido, reglas e interfaz (necesario para RNF01 y el riesgo R1). La decisión se documenta con alternativas y consecuencias en la Unidad 6.